



计算机系统原理实验报告

第四次实验

基于 RISC-V 开发板的加密算法性能测试与加速比分析

成员：	江李阳	202400460042
	王健一	202400460037
	邱泽辉	202400460046
	史淇	202333180074
专业：	密码科学与技术	
日期：	June 11, 2026	

Contents

1	实验目的	2
2	实验环境	2
2.1	开发板硬件环境	2
2.2	开发板软件环境	2
2.3	主机环境	3
3	实验原理	3
3.1	SM4 算法	3
3.1.1	加密算法	3
3.1.2	轮密钥生成方案	4
3.1.3	解密算法	4
4	实验过程	4
4.1	实验环境搭建	4
4.2	SM4 示例程序的测试使用	6
4.3	SM3 示例程序的使用测试	7
4.4	SM4 默认循环与循环展开	7
4.4.1	SM4 默认循环	7
4.4.2	SM4 循环展开	8
4.5	SM4 默认 XBox 查表与 XBox 合并	9
4.5.1	SM4 默认 XBox 查表	10
4.5.2	XBox 合并	11
4.6	SM4 默认轮函数连续异或与减少指令依赖	13
4.7	默认字节序转换与 RISC-V B 拓展 rev8 转换	14
5	性能测试对比	17
6	实验结果分析	20
6.1	总体性能对比	20
6.2	查表优化为何有效	20
6.3	微架构优化的有限效果	21
6.3.1	循环展开 (unrolled)	21
6.3.2	降低指令依赖 (reduced_dep)	21
6.4	ISA 扩展的独立与叠加效应	22
6.4.1	独立应用 Zbb 扩展	22
6.4.2	叠加应用 Zbb 扩展	23
6.5	数据规模、缓存与局部性	23
7	实验总结	25

1 实验目的

1. 学习并掌握 RISC-V 架构开发板 (香橙派) 的基本使用方法。
2. 学习 OpenHiTLS 加密算法的使用示例, 能正确运行 SM3 或 SM4 实例。
3. 记录各方案的运行时间, 吞吐率以及加速比。
4. 结合技术方案差异、开发板硬件条件、系统环境和测试方法等, 对实验结果进行分析。
5. 若某一技术方案依赖开发板不支持的指令扩展, 应说明原因并选择替代方案。

2 实验环境

2.1 开发板硬件环境

- 开发板型号: Orange Pi RV2
- 处理器: Ky(R) X1 (RISC-V 64 位)
 - 架构: riscv64 (Little Endian)
 - ISA 扩展: rv64imafdcv_zicsr_zifencei_zihintpause_zba_zbb_zbc_zbs_zicbop_zicboz
 - 处理器核心: 8 核 (1 Socket $\times$ 8 Cores)
 - 每核心线程数: 1
 - CPU 频率范围: 614.4 MHz (最低) — 1600.0 MHz (最高)
- 缓存:
 - L1d Cache: 256 KiB (8 实例, 即每核心 32 KiB)
 - L1i Cache: 256 KiB (8 实例, 即每核心 32 KiB)
 - L2 Cache: 1 MiB (2 实例, 即每 4 核心共享 512 KiB)
- 内存: 7.7 GiB (无 Swap)

2.2 开发板软件环境

- 操作系统: Ubuntu 24.04
- 交叉编译器: riscv64-linux-gnu-gcc / g++ 13.3.0
- 构建工具: CMake 3.28.3、GNU Make 4.3
- OpenHiTLS: 实验仓库内置源码快照 (仓库 commit: 6da83cf)
- libboundscheck: 实验仓库内置源码快照 (仓库 commit: 6da83cf)

2.3 主机环境

- 主机硬件
 - 处理器:13th Gen Intel(R) Core(TM) i5-13420H
 - 内存:16 GB
- 主机软件
 - CP210x Windows Drivers(USB 转 TTL 串口芯片驱动)
 - MobaXterm(终端管理工具)
 - OpenSSH_for_Windows_9.5p2, LibreSSL 3.8.2(远程访问开发板)

3 实验原理

3.1 SM4 算法

SM4 是我国商用密码标准 (GB/T 32907-2016) 中规定的分组密码算法, 采用非平衡 Feistel 结构, 分组长度为 128 bit, 密钥长度为 128 bit, 迭代轮数为 32 轮。

3.1.1 加密算法

输入 128bit 明文 m , 按 32-bit 分为 4 个字 $X_0^0, X_0^1, X_0^2, X_0^3$ 进行 32 轮迭代, 每轮迭代都有一个 32bit 的轮密钥 K_{i-1} 参与计算, 第 $i (i = 1, 2, \dots, 32)$ 轮的计算如下:

$$\begin{aligned} X_i^0 &= X_{i-1}^1 \\ X_i^1 &= X_{i-1}^2 \\ X_i^2 &= X_{i-1}^3 \\ X_i^3 &= X_{i-1}^0 \oplus T(X_{i-1}^1 \oplus X_{i-1}^2 \oplus X_{i-1}^3 \oplus K_{i-1}) \end{aligned}$$

对最后一轮迭代的输出进行反序变化 R 得到 128bit 密文 c :

$$c = (Y^0, Y^1, Y^2, Y^3) = R(X_{32}^0, X_{32}^1, X_{32}^2, X_{32}^3) = (X_{32}^3, X_{32}^2, X_{32}^1, X_{32}^0)$$

其中函数 $T: \mathbb{F}_2^{32} \rightarrow \mathbb{F}_2^{32}$ 由非线性变换 S 和线性变换 L 组成:

$$T = L \circ S$$

设函数 T 的 32bit 输入为 $A = (a_0, a_1, a_2, a_3) \in (\mathbb{F}_{2^8})^4$, 则

- 非线性变换 S : 对每个字节单独进行查表代替操作, 即

$$B = (S(a_0), S(a_1), S(a_2), S(a_3))$$

类似 AES 加密, SM4 算法采用 1 个 S 盒, 查表时将每个字节的高 4bit 看作行标, 低 4bit 看作列标, 对应取值即为输出字节。

- 线性变换 L : 对非线性变换 S 的 32bit 输出 B , 计算

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$$

其中 $\lll x$ 表示循环左移 x bit。

3.1.2 轮密钥生成方案

SM4 算法的主密钥 K 为 128bit, 轮密钥生成方案也采用非平衡 Feistel 结构。轮密钥 K_i 由主密钥 K 通过密钥扩展算法生成, 具体做法为: 先将主密钥 K 按 32-bit 分为 4 个字 MK_0, MK_1, MK_2, MK_3 , 然后与 32-bit 常量 FK_i 进行异或得到初始轮密钥:

$$K_{-4} = MK_0 \oplus FK_0$$

$$K_{-3} = MK_1 \oplus FK_1$$

$$K_{-2} = MK_2 \oplus FK_2$$

$$K_{-1} = MK_3 \oplus FK_3$$

其中 $FK_0 = 0xA3B1BAC6, FK_1 = 0x56AA3350, FK_2 = 0x677D9197, FK_3 = 0xB27022DC$ 。接着通过迭代计算得到后续轮密钥:

$$K_i = K_{i-4} \oplus T'(K_{i-3} \oplus K_{i-2} \oplus K_{i-1} \oplus CK_i), \quad i = 0, 1, \dots, 31$$

其中 CK_i 为固定参数; T' 函数与 T 函数类似, 但线性变换部分不同, 仅需替换 L 为 L' :

$$T'(X) = L'(S(X))$$

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$$

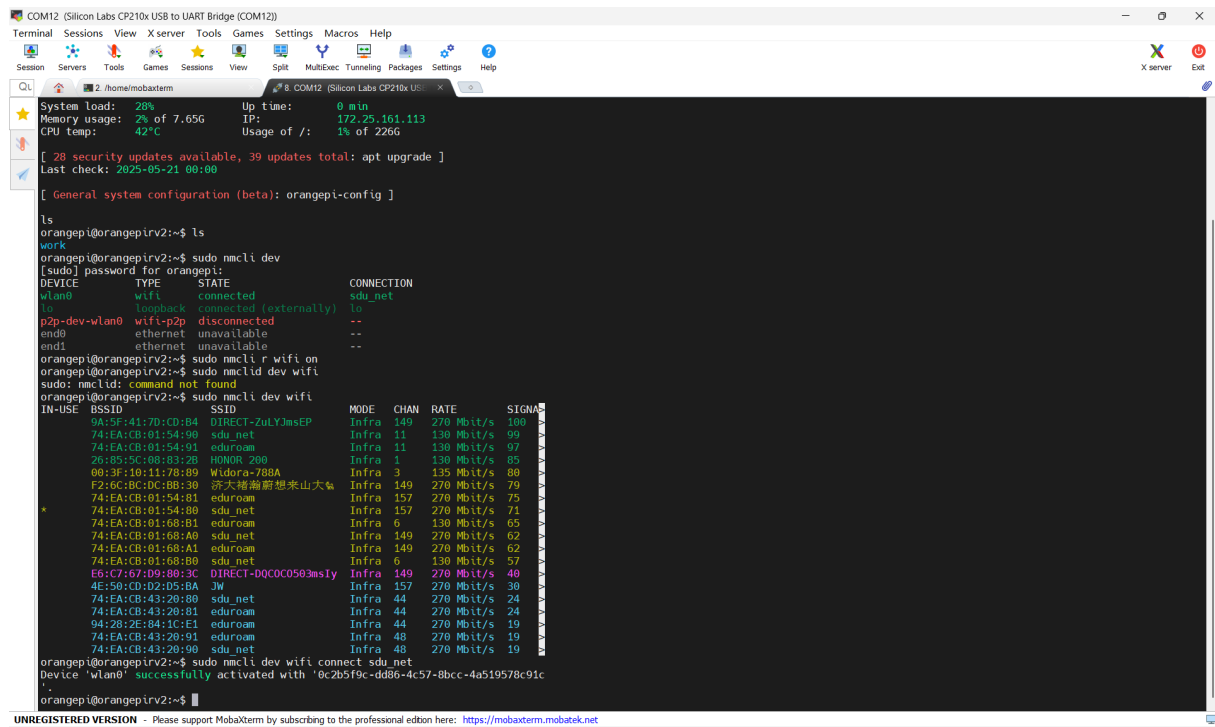
3.1.3 解密算法

因为采用广义 Feistel 结构, 所以 SM4 算法具有加解密结构一致性, 因此解密算法与加密算法一致, 仅需将参与第 i ($i = 1, 2, \dots, 32$) 轮迭代运算的轮密钥由 K_{i-1} 替换为 K_{32-i} 即可。

4 实验过程

4.1 实验环境搭建

为香橙派开发板插上 TYPEC 电源适配器并用 USB 转 TTL 模块连接开发板与主机, 连接完成后使用 MobaXterm 访问开发板, 终端连接 WIFI 成功如图:



```
COM12 (Silicon Labs CP210x USB to UART Bridge (COM12))
Terminal Sessions View Xserver Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help

System load: 20% Up time: 0 min
Memory usage: 2% of 7.65G IP: 172.25.161.113
CPU temp: 42°C Usage of /: 1% of 228G

[ 28 security updates available, 39 updates total: apt upgrade ]
Last check: 2025-05-21 00:00

[ General system configuration (beta): orangepi-config ]

ls
orangepi@orangepi2:~$ ls
work
orangepi@orangepi2:~$ sudo nmcli dev
[sudo] password for orangepi:
DEVICE TYPE STATE CONNECTION
wlan0 wifi connected (externally) sdu_net
lo loopback connected (externally) lo
p2p-dev-wlan0 wifi-p2p disconnected --
end0 ethernet unavailable --
orangepi@orangepi2:~$ sudo nmcli r wifi on
orangepi@orangepi2:~$ sudo nmcli dev wifi
sudo: nmcli: command not found
orangepi@orangepi2:~$ sudo nmcli dev wifi
IN-USE BSSID SSID MODE CHAN RATE SIGNAL
74:EA:CB:01:54:90 sdu_net Infra 11 130 Mbit/s 99
74:EA:CB:01:54:91 eduroam Infra 11 130 Mbit/s 97
20:89:5C:08:83:2B HONOR 200 Infra 1 130 Mbit/s 85
00:3F:10:11:70:09 Widora-700A Infra 3 135 Mbit/s 80
F2:6C:8C:DC:8B:30 齐大格瑞斯想来山大学 Infra 149 270 Mbit/s 79
74:EA:CB:01:54:81 eduroam Infra 157 270 Mbit/s 75
74:EA:CB:01:54:80 sdu_net Infra 157 270 Mbit/s 71
74:EA:CB:01:68:B1 eduroam Infra 6 130 Mbit/s 65
74:EA:CB:01:68:A0 sdu_net Infra 149 270 Mbit/s 62
74:EA:CB:01:68:A1 eduroam Infra 140 270 Mbit/s 62
74:EA:CB:01:68:B0 sdu_net Infra 6 130 Mbit/s 57
E6:C7:67:D9:80:3C DIRECT-DQCO0503msly Infra 149 270 Mbit/s 48
4E:58:CD:D2:D5:BA JW Infra 157 270 Mbit/s 38
74:EA:CB:43:20:80 sdu_net Infra 44 270 Mbit/s 24
74:EA:CB:43:20:81 eduroam Infra 44 270 Mbit/s 24
94:2B:2E:84:1C:E1 eduroam Infra 44 270 Mbit/s 19
74:EA:CB:43:20:91 eduroam Infra 48 270 Mbit/s 19
74:EA:CB:43:20:90 sdu_net Infra 48 270 Mbit/s 19
orangepi@orangepi2:~$ sudo nmcli dev wifi connect sdu_net
Device 'wlan0' successfully activated with '0c2b5f9c-dd86-4c57-8bcc-4a519578c91c'.
orangepi@orangepi2:~$
```

Figure 1: 开发板连接 WIFI

考虑到原实验为串口连接开发板进行单人次实验, 而实际实验小组有多人, 轮流使用串口会显著增加实验总耗时, 不方便调试, 也不便于课后小组成员随时随地使用开发板的需要, 因此我们为开发板配置网络后启动了 OpenSSH 服务。基本实现为借助 Tailscale 工具将包括开发板在内的所有主机添加到同一个虚拟局域网中, 使得所有设备都可以跨局域网登录访问开发板:

```
orangepi@orangepi2:~$ ssh orangepi@100.85.227.127
orangepi@100.85.227.127's password:

Welcome to Orange Pi 1.0.0 Noble with Linux 6.6.63-ky

System load: 26%      Up time: 3 days 9:37      Local users: 4
Memory usage: 3% of 7.65G      IP: 100.85.227.127 10.104.66.182
CPU temp: 47°C      Usage of /: 2% of 226G

[ 147 security updates available, 210 updates total: apt upgrade ]
Last check: 2026-05-26 00:01

[ General system configuration (beta): orangepi-config ]

Last login: Tue May 26 11:38:29 2026 from 100.72.212.17
orangepi@orangepi2:~$ who
orangepi ttyS0      2026-05-23 02:38
orangepi tty1      2026-05-23 02:38
orangepi pts/0      2026-05-26 07:42 (100.118.90.106)
orangepi pts/1      2026-05-26 12:15 (100.72.212.17)
orangepi@orangepi2:~$
```

Figure 2: 远程连接开发板

这里远程连接只是为了方便多用户远程访问开发板和编写代码, 为确保实验结果准确性, 每次测试算法效率时仅一位用户在使用。

4.2 SM4 示例程序的测试使用

按照实验指导书的指示分别安装依赖、获取 libboundscheck 源码并编译、获取 openhitls 源码并编译、然后运行实例代码, 结果如下图:

```
orangepi@orangepi2:~/work/openhitls/testcode/demo/build$ ls
client      CMakeFiles      drbg  hash  pbkdf2  server  sm2sign  sm4cbc_origin
CMakeCache.txt  cmake_install.cmake  ecdh  Makefile  privpass_token  sm2enc  sm4cbc  sm4_default_loop
orangepi@orangepi2:~/work/openhitls/testcode/demo/build$ cmake .. -DCMAKE_CXX_COMPILER=riscv64-linux-g
nu-gcc
statusHITLS_ROOT:
-- Configuring done (0.1s)
-- Generating done (0.2s)
-- Build files have been written to: /home/orangepi/work/openhitls/testcode/demo/build
orangepi@orangepi2:~/work/openhitls/testcode/demo/build$ make
[ 8%] Built target client
[ 16%] Built target server
[ 25%] Built target drbg
[ 33%] Built target ecdh
[ 41%] Built target pbkdf2
[ 50%] Built target sm2enc
[ 58%] Built target sm2sign
[ 66%] Built target sm4cbc
[ 75%] Built target hash
[ 83%] Built target privpass_token
[ 91%] Built target sm4cbc_origin
[100%] Built target sm4_default_loop
orangepi@orangepi2:~/work/openhitls/testcode/demo/build$ ./sm4cbc
Encryption took 28361 microseconds
Decryption took 21096 microseconds
pass
orangepi@orangepi2:~/work/openhitls/testcode/demo/build$
```

Figure 3: SM4 示例程序测试运行

4.3 SM3 示例程序的使用测试

修改 hash.c 文件中的部分代码, 重新编译即可测试运行 SM3 示例程序, 结果如下:

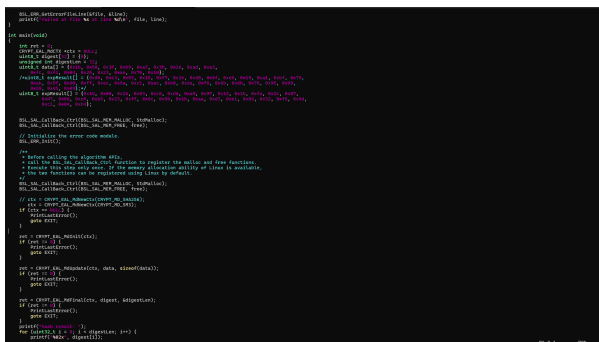
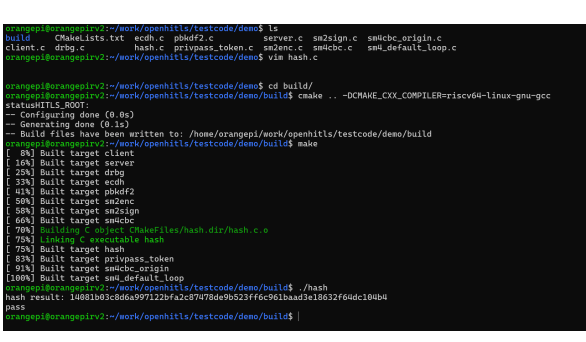



Figure 4: 修改代码

Figure 5: sm3 示例程序测试运行

4.4 SM4 默认循环与循环展开

4.4.1 SM4 默认循环

按照课本介绍 SM4 算法的实现, 不采用任何优化策略, 完整实现 SM4 的 32 轮迭代并在每一轮计算中对明文分组做非线性变换 S 和线性变换 L, 记录算法运行时间, 并以此组作为基准组, 时间开销主要在 32 轮迭代以及每轮迭代中的 S 和 L 操作:

openhits/testcode/demo/sm4_default_loop.c

```
1 static void sm4EncryptBlock(const uint32_t rk[32],
2                             const uint8_t in[16],
3                             uint8_t out[16])
4 {
5     uint32_t X[4];
6     sm4LoadBlock(in, X);
7
8     for (int i = 0; i < 32; i++) {
9         uint32_t tmp = X[1] ^ X[2] ^ X[3] ^ rk[i];
10        uint32_t t = sm4T(tmp);
11        uint32_t newX0 = X[0] ^ t;
12        X[0] = X[1];
13        X[1] = X[2];
14        X[2] = X[3];
15        X[3] = newX0;
16    }
17
18    sm4StoreBlock(X, out);
19 }
```

结果如下:


```

orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_default_loop
[PASS] sm4_default_loop: test vector correct
SM4 Default Loop - 1048576 bytes x 10 repeats
Elapsed : 445129 µs
Throughput: 22.465 MB/s
orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$

```

其中测试输入数据大小为 1MB。

4.4.2 SM4 循环展开

上述的循环次数以及每次循环的操作是已知的, 使用宏一次性生成全部 32 轮代码, 可以去掉 32 次 `i++` 和条件判断, 避免分支预测失败惩罚, 同时再使用数组而是四个变量即可实现 32 轮循环迭代, 省去计算数组索引和基于偏移量取数的开销, 关键代码如下:

openhitls/testcode/demo/sm4_unrolled.c

```

1 static void sm4EncryptBlock(const uint32_t rk[32],
2                             const uint8_t in[16],
3                             uint8_t out[16])
4 {
5     uint32_t X0, X1, X2, X3;
6     uint32_t tmp, t, newX;
7
8     /* Manual load to scalars for unrolled access */
9     {
10         uint32_t arr[4];
11         sm4LoadBlock(in, arr);
12         X0 = arr[0]; X1 = arr[1]; X2 = arr[2]; X3 = arr[3];
13     }
14
15 #define SM4_UNROLLED_ROUND(N) do { \
16     tmp = X1 ^ X2 ^ X3 ^ rk[N]; \
17     t = sm4T(tmp); \
18     newX = X0 ^ t; \
19     X0 = X1; X1 = X2; X2 = X3; X3 = newX; \
20 } while(0)
21
22 SM4_UNROLLED_ROUND(0);
23 SM4_UNROLLED_ROUND(1);
24 SM4_UNROLLED_ROUND(2);
25 SM4_UNROLLED_ROUND(3);
26 SM4_UNROLLED_ROUND(4);
27 SM4_UNROLLED_ROUND(5);
28 SM4_UNROLLED_ROUND(6);
29 SM4_UNROLLED_ROUND(7);
30 SM4_UNROLLED_ROUND(8);
31 SM4_UNROLLED_ROUND(9);
32 SM4_UNROLLED_ROUND(10);
33 SM4_UNROLLED_ROUND(11);
34 SM4_UNROLLED_ROUND(12);
35 SM4_UNROLLED_ROUND(13);

```

```

36     SM4_UNROLLED_ROUND(14);
37     SM4_UNROLLED_ROUND(15);
38     SM4_UNROLLED_ROUND(16);
39     SM4_UNROLLED_ROUND(17);
40     SM4_UNROLLED_ROUND(18);
41     SM4_UNROLLED_ROUND(19);
42     SM4_UNROLLED_ROUND(20);
43     SM4_UNROLLED_ROUND(21);
44     SM4_UNROLLED_ROUND(22);
45     SM4_UNROLLED_ROUND(23);
46     SM4_UNROLLED_ROUND(24);
47     SM4_UNROLLED_ROUND(25);
48     SM4_UNROLLED_ROUND(26);
49     SM4_UNROLLED_ROUND(27);
50     SM4_UNROLLED_ROUND(28);
51     SM4_UNROLLED_ROUND(29);
52     SM4_UNROLLED_ROUND(30);
53     SM4_UNROLLED_ROUND(31);
54
55 #undef SM4_UNROLLED_ROUND
56
57     /* Reverse transform & store */
58     {
59         uint32_t arr[4] = {X0, X1, X2, X3};
60         sm4StoreBlock(arr, out);
61     }
62 }

```

这里所有宏会在编译阶段就计算出来, 能降低运行时计算的时间开销。运行结果如下:

```

orange@orangeiprv2: ~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ls
sm4_default_loop  sm4_reduced_dep  sm4_unrolled  sm4_xbox  sm4_xbox_merged  sm4_xbox_zbb  sm4_zbb
orange@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_unrolled
[PASS] sm4_unrolled: test vector correct
SM4 Loop Unrolled - 1048576 bytes x 10 repeats
Elapsed : 405184 µs
Throughput: 24.680 MB/s
orange@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$

```

Figure 6: SM4 循环展开

可以看到循环展开能有效提升 SM4 加密算法的运行效率, 提升吞吐量。

4.5 SM4 默认 XBox 查表与 XBox 合并

在 SM4 加密算法的每轮循环中需要对输入的每个字做非线性的过 S 盒操作, 和线性的 L 操作, 对于

$$X_3 = a_0 || a_1 || a_2 || a_3$$

为最后一个 32bit 分组, 分别过 S 盒后得到

$$B = b_0 || b_1 || b_2 || b_3 = Sbox(a_0) || Sbox(a_1) || Sbox(a_2) || Sbox(a_3)$$

在代码中通过移位拼接:

$$B = (b_0 \ll 24) \oplus (b_1 \ll 16) \oplus (b_2 \ll 8) \oplus (b_3)$$

由于 L 是线性操作, 满足

$$L(a \oplus b) = L(a) \oplus L(b)$$

因此

$$\begin{aligned} L(B) &= L(b_0 \ll 24) \oplus L(b_1 \ll 16) \oplus L(b_2 \ll 8) \oplus L(b_3) \\ &= B \oplus \text{ROL}(B, 2) \oplus \text{ROL}(B, 10) \oplus \text{ROL}(B, 18) \oplus \text{ROL}(B, 24) \\ &= \text{ROL}(\text{Xbox}[b_0], 24) \oplus \text{ROL}(\text{Xbox}[b_1], 16) \oplus \text{ROL}(\text{Xbox}[b_2], 8) \oplus \text{Xbox}[b_3] \end{aligned} \quad (1)$$

4.5.1 SM4 默认 Xbox 查表

因此预计算 $\text{Xbox}[i] = L(S(i))$ 共 $2^8 = 256$ 项, 最后循环移位并异或即可得到最终结果。关键代码如下:

openhits/testcode/demo/sm4_xbox.c

```
1 static void sm4EncryptBlock(const uint32_t rk[32],
2                             const uint8_t in[16],
3                             uint8_t out[16])
4 {
5     uint32_t X0, X1, X2, X3;
6     {
7         uint32_t arr[4];
8         sm4LoadBlock(in, arr);
9         X0 = arr[0]; X1 = arr[1]; X2 = arr[2]; X3 = arr[3];
10    }
11
12    initXbox();
13
14    for (int i = 0; i < 32; i++) {
15        uint32_t t = X1 ^ X2 ^ X3 ^ rk[i];
16        uint8_t b0 = (uint8_t)((t >> 24) & 0xFF);
17        uint8_t b1 = (uint8_t)((t >> 16) & 0xFF);
18        uint8_t b2 = (uint8_t)((t >> 8) & 0xFF);
19        uint8_t b3 = (uint8_t)(t & 0xFF);
20
21        uint32_t newX0 = X0 ^ ROL(xbox[b0], 24)
22                        ^ ROL(xbox[b1], 16)
23                        ^ ROL(xbox[b2], 8)
24                        ^ xbox[b3];
25
26        X0 = X1; X1 = X2; X2 = X3; X3 = newX0;
27    }
28
29    {
30        uint32_t arr[4] = {X0, X1, X2, X3};
```

```

31         sm4StoreBlock(arr, out);
32     }
33 }

```

编译运行结果截图如下:

```

orangepi@orangepirv2: ~/Prin
orangepi@orangepirv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ls
sm4_default_loop sm4_reduced_dep sm4_unrolled sm4_xbox sm4_xbox_merged sm4_xbox_zbb sm4_zbb
orangepi@orangepirv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_xbox
[PASS] sm4_xbox: test vector correct
SM4 Xbox Table Lookup - 1048576 bytes x 10 repeats
Elapsed : 365158 μs
Throughput: 27.385 MB/s

```

Figure 7: SM4 默认 Xbox

效率较 SM4 默认循环和循环展开都有所提升, 不过这种提前计算会增大程序的空间复杂度。

4.5.2 Xbox 合并

观察 (1) 式可以发现最后循环移位的位数是固定的 (分别为 24、16、8、0), 可以分别预计算这里的固定位数循环移位, 将原本运行时计算 ROL 改为编译时计算且运行时查表, 实现 Xbox 合并。即计算:

$$\begin{cases} XBox0[i] = L(S(i)) \\ XBox1[i] = ROL(L(S(i)), 8) \\ XBox2[i] = ROL(L(S(i)), 16) \\ XBox3[i] = ROL(L(S(i)), 24) \end{cases}$$

那么实际运行时各个位置查对应的表即可:

$$\begin{cases} ROL(Xbox[b_0], 24) = XBox3[b_0] \\ ROL(Xbox[b_1], 16) = XBox2[b_1] \\ ROL(Xbox[b_2], 8) = XBox1[b_2] \end{cases}$$

关键代码如下:

```

openhitls/testcode/demo/sm4_xbox_merged.c

1 static uint32_t XBOX0[256];
2 static uint32_t XBOX1[256];
3 static uint32_t XBOX2[256];
4 static uint32_t XBOX3[256];
5 static int xbox_merged_initialized = 0;
6
7 static inline uint32_t ROL(uint32_t x, int n)
8 {
9     return (x << n) | (x >> (32 - n));
10 }

```

```
11
12 static void initXboxMerged(void)
13 {
14     if (xbox_merged_initialized) return;
15
16     for (int i = 0; i < 256; i++) {
17         XBOX0[i] = sm4L((uint32_t)sm4Sbox((uint8_t)i));
18         XBOX1[i] = ROL(XBOX0[i], 8);
19         XBOX2[i] = ROL(XBOX0[i], 16);
20         XBOX3[i] = ROL(XBOX0[i], 24);
21     }
22     xbox_merged_initialized = 1;
23 }
24
25 static void sm4EncryptBlock(const uint32_t rk[32],
26                             const uint8_t in[16],
27                             uint8_t out[16])
28 {
29     uint32_t X0, X1, X2, X3;
30     {
31         uint32_t arr[4];
32         sm4LoadBlock(in, arr);
33         X0 = arr[0]; X1 = arr[1]; X2 = arr[2]; X3 = arr[3];
34     }
35
36     initXboxMerged();
37
38     for (int i = 0; i < 32; i++) {
39         uint32_t t = X1 ^ X2 ^ X3 ^ rk[i];
40         uint32_t newX0 = X0 ^ XBOX3[(t >> 24) & 0xFF]
41                         ^ XBOX2[(t >> 16) & 0xFF]
42                         ^ XBOX1[(t >> 8) & 0xFF]
43                         ^ XBOX0[t & 0xFF];
44
45         X0 = X1; X1 = X2; X2 = X3; X3 = newX0;
46     }
47
48     {
49         uint32_t arr[4] = {X0, X1, X2, X3};
50         sm4StoreBlock(arr, out);
51     }
52 }
```

编译运行后结果如下:

```

orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ls
sm4_default_loop sm4_reduced_dep sm4_unrolled sm4_xbox sm4_xbox_merged sm4_xbox_zbb sm4_zbb
orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_xbo
x_merged
[PASS] sm4_xbox_merged: test vector correct
SM4 XBox Merged (4 tables) - 1048576 bytes x 10 repeats
Elapsed : 284581 μs
Throughput: 35.139 MB/s
orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ |

```

Figure 8: SM4XBox 合并

可以看到程序运行效率和吞吐量相较之前版本优化得到很大提升,但因为需要预计算和存储 4 张表,因此有总计 $4 * 256 * 4B = 4KB$ 的额外空间开销。

4.6 SM4 默认轮函数连续异或与减少指令依赖

在默认循环实现版本中每轮循环都有 $tmp = X1 \oplus X2 \oplus X3 \oplus rk[i]$, 由于异或操作满足结合律:

$$a \oplus b \oplus c \oplus d = (a \oplus b) \oplus (c \oplus d)$$

因此这里可以分别计算前两个变量的异或结果和后两个变量的异或结果, 分别存到临时变量中, 最后将这两个临时变量再求异或, 这样能减少指令之间的依赖性, 即可以同时求两个临时变量。否则, 连续异或时想要求后一个异或必须要等前一个异或运算结束, 指令依赖关系导致只能串行执行。在 SM4 每轮最后的换位赋值时可能产生写寄存器又马上读的依赖, 因此可以用临时变量存放 X_1, X_2 值, 这样能确保整个过程只发生寄存器移动而无需内存访问。具体代码如下:

openhitls/testcode/demo/sm4_reduced_dep.c

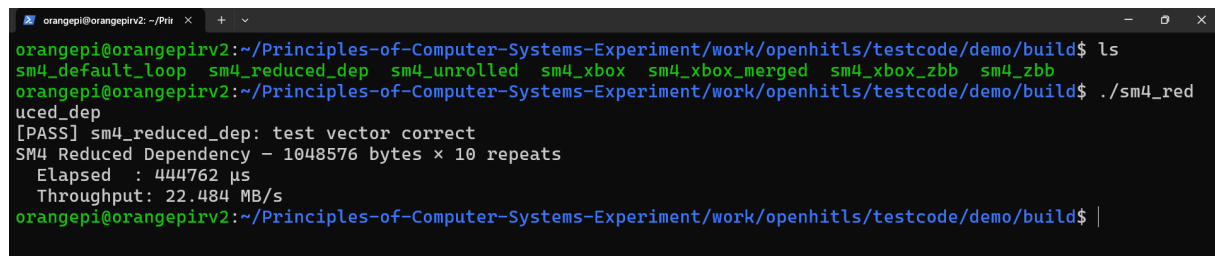
```

1 static void sm4EncryptBlock(const uint32_t rk[32],
2                             const uint8_t in[16],
3                             uint8_t out[16])
4 {
5     uint32_t X0, X1, X2, X3;
6     {
7         uint32_t arr[4];
8         sm4LoadBlock(in, arr);
9         X0 = arr[0]; X1 = arr[1]; X2 = arr[2]; X3 = arr[3];
10    }
11
12    for (int i = 0; i < 32; i++) {
13        /* Break the XOR chain: X1^X2 and X3^rk are independent */
14        uint32_t tmp1 = X1 ^ X2;
15        uint32_t tmp2 = X3 ^ rk[i];
16        uint32_t tmp = tmp1 ^ tmp2;
17        uint32_t t = sm4T(tmp);
18        uint32_t newX0 = X0 ^ t;
19
20        /* Feistel rotation with temporaries */
21        uint32_t prevX1 = X1;

```

```
22     uint32_t prevX2 = X2;
23     X0 = prevX1;
24     X1 = prevX2;
25     X2 = X3;
26     X3 = newX0;
27 }
28
29 {
30     uint32_t arr[4] = {X0, X1, X2, X3};
31     sm4StoreBlock(arr, out);
32 }
33 }
```

编译运行结果如图:



```
orangeipi@orangeiprv2: ~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ls
sm4_default_loop sm4_reduced_dep sm4_unrolled sm4_xbox sm4_xbox_merged sm4_xbox_zbb sm4_zbb
orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_reduced_dep
[PASS] sm4_reduced_dep: test vector correct
SM4 Reduced Dependency - 1048576 bytes x 10 repeats
Elapsed   : 444762 µs
Throughput: 22.484 MB/s
orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ |
```

Figure 9: SM4 减少指令依赖

可以看到程序运行效率和吞吐量较默认循环版几乎相同, 优化非常有限。

4.7 默认字节序转换与 RISC-V B 拓展 rev8 转换

查看开发板 CPU 所支持的指令集如下图:

```

orangeip1@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhits/testcode/demo/build$ cat /proc/cpuinfo
processor       : 0
hart           : 0
model name     : Ky(R) X1
isa            : rv64imafdcv_zicbom_zicboz_zicntr_zicnd_zicsr_zifencei_zihintpause_zihpm_zfh_zfhmin_zca_zcd_zba_zbb_zbc_zbs_zkt_zve32f_zve32x_
zve64d_zve64f_zve64x_zvfh_zvfhmin_zvkt_zscofpmf_zstc_zvinval_zvnapot_zvpbmt
mmu            : sv39
uarch          : ky, x60
mvendorid      : 0x710
marchid        : 0x8000000058000001
mimpid         : 0x1000000049772200

processor       : 1
hart           : 1
model name     : Ky(R) X1
isa            : rv64imafdcv_zicbom_zicboz_zicntr_zicnd_zicsr_zifencei_zihintpause_zihpm_zfh_zfhmin_zca_zcd_zba_zbb_zbc_zbs_zkt_zve32f_zve32x_
zve64d_zve64f_zve64x_zvfh_zvfhmin_zvkt_zscofpmf_zstc_zvinval_zvnapot_zvpbmt
mmu            : sv39
uarch          : ky, x60
mvendorid      : 0x710
marchid        : 0x8000000058000001
mimpid         : 0x1000000049772200

processor       : 2
hart           : 2
model name     : Ky(R) X1
isa            : rv64imafdcv_zicbom_zicboz_zicntr_zicnd_zicsr_zifencei_zihintpause_zihpm_zfh_zfhmin_zca_zcd_zba_zbb_zbc_zbs_zkt_zve32f_zve32x_
zve64d_zve64f_zve64x_zvfh_zvfhmin_zvkt_zscofpmf_zstc_zvinval_zvnapot_zvpbmt
mmu            : sv39
uarch          : ky, x60
mvendorid      : 0x710
marchid        : 0x8000000058000001
mimpid         : 0x1000000049772200

processor       : 3
hart           : 3
model name     : Ky(R) X1
isa            : rv64imafdcv_zicbom_zicboz_zicntr_zicnd_zicsr_zifencei_zihintpause_zihpm_zfh_zfhmin_zca_zcd_zba_zbb_zbc_zbs_zkt_zve32f_zve32x_
zve64d_zve64f_zve64x_zvfh_zvfhmin_zvkt_zscofpmf_zstc_zvinval_zvnapot_zvpbmt
mmu            : sv39
uarch          : ky, x60

```

Figure 10: 开发板 CPU 信息

可以看到该板支持 zbb, 因此可以在编译时启用该拓展。该拓展的优化原理在于:

- 对字节序转换的优化: 标准 SM4 中对串 01234567 的解释为:

01 23 45 67

即大端序, 而在 RISC-V 中却是小端序, 因此 sm4LoadBlock() 函数需要转换。普通实现时的代码

```

1 x =
2 (in[0]<<24)
3 |
4 (in[1]<<16)
5 |
6 (in[2]<<8)
7 |
8 in[3];

```

对应的汇编代码为:

```

1 lbu
2 slli
3
4 lbu
5 slli
6
7 lbu
8 slli

```



```

9
10 or
11 or
12 or

```

需要很多条指令才能完成, 而使用 Zbb 拓展中的 rev8 指令则可直接完成这一功能, 即直接

```

1 lw
2 rev8

```

即可完成, 使得如 sm4LoadBlock()、sm4StoreBlock() 函数得到加速。

- Zbb 对 L 变换的优化: 针 ROL(x, 10) 操作, 编译器默认生成:

```

1 slli t1,x,10
2 srli t2,x,22
3 or   t1,t1,t2

```

即:

$$(x \ll 10) | (x \gg 22)$$

这需要 2 次 shift 和 1 次 or 共 3 条指令, 对于 SM4 每轮的四次 ROL 则总共需要 12 条指令。Zbb 拓展提供 rori(Rotate Right Immediate) 指令, 可直接实现一次 ROL 操作, 因此每轮 L 变换仅需 4 条指令即可完成。

对比代码不同编译方式运行结果如下:

```

orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo$ riscv64-linux-gnu-gcc \
-O2 \
-march=rv64gc \
-Wall -Wextra -std=c99 \
-D_POSIX_C_SOURCE=199309L \
-o sm4_zbb_nozbb \
sm4_zbb.c
orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo$ ./sm4_zbb_nozbb
[PASS] sm4_zbb: test vector correct
SM4 Default Loop + Zbb - 1048576 bytes x 10 repeats
Elapsed : 444762 µs
Throughput: 22.484 MB/s
orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo$ riscv64-linux-gnu-gcc \
-O2 \
-march=rv64gc_zba_zbb_zbc_zbs \
-Wall -Wextra -std=c99 \
-D_POSIX_C_SOURCE=199309L \
-o sm4_zbb_zbb \
sm4_zbb.c
orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo$ ./sm4_zbb_zbb
[PASS] sm4_zbb: test vector correct
SM4 Default Loop + Zbb - 1048576 bytes x 10 repeats
Elapsed : 392394 µs
Throughput: 25.485 MB/s
orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo$

```

Figure 11: SM4 默认字节序转换与 rev8 转换

可以看到拓展指令能小幅提升程序效率与吞吐量。

5 性能测试对比

针对上述 7 种 SM4 实现方案, 分别在 7 种不同输入数据规模 (1 KB 至 16 MB) 下测试程序的运行时间并计算吞吐率与加速比, 过程截图如下:

Data sizes: 1 KB, 16 KB, 64 KB, 256 KB, 1 MB, 4 MB, 16 MB				
--- 1 KB (1 KB, ×2000 repeats) ---				
Running default_loop	...	88985 μs		21.949 MB/s
Running unrolled	...	78646 μs		24.834 MB/s
Running reduced_dep	...	88667 μs		22.028 MB/s
Running xbox	...	68855 μs		28.366 MB/s
Running xbox_merged	...	53654 μs		36.402 MB/s
Running zbb_default	...	78304 μs		24.943 MB/s
Running zbb_xbox	...	54257 μs		35.998 MB/s
--- 16 KB (16 KB, ×200 repeats) ---				
Running default_loop	...	142619 μs		21.912 MB/s
Running unrolled	...	126819 μs		24.641 MB/s
Running reduced_dep	...	142404 μs		21.945 MB/s
Running xbox	...	111273 μs		28.084 MB/s
Running xbox_merged	...	86488 μs		36.132 MB/s
Running zbb_default	...	126302 μs		24.742 MB/s
Running zbb_xbox	...	87457 μs		35.732 MB/s
--- 64 KB (64 KB, ×50 repeats) ---				
Running default_loop	...	142590 μs		21.916 MB/s
Running unrolled	...	126666 μs		24.671 MB/s
Running reduced_dep	...	142267 μs		21.966 MB/s
Running xbox	...	110963 μs		28.163 MB/s
Running xbox_merged	...	86442 μs		36.151 MB/s
Running zbb_default	...	126072 μs		24.787 MB/s
Running zbb_xbox	...	87641 μs		35.657 MB/s
--- 256 KB (256 KB, ×20 repeats) ---				
Running default_loop	...	228371 μs		21.894 MB/s
Running unrolled	...	203515 μs		24.568 MB/s
Running reduced_dep	...	228545 μs		21.878 MB/s
Running xbox	...	178726 μs		27.976 MB/s
Running xbox_merged	...	140590 μs		35.564 MB/s
Running zbb_default	...	202422 μs		24.701 MB/s
Running zbb_xbox	...	141695 μs		35.287 MB/s

Figure 12: 对比测试各种优化方案效率 (1 KB 至 256 KB)

Running xbox_merged	...	140590 μ s		35.564 MB/s
Running zbb_default	...	202422 μ s		24.701 MB/s
Running zbb_xbox	...	141695 μ s		35.287 MB/s
--- 1 MB (1 MB, x5 repeats) ---				
Running default_loop	...	229836 μ s		21.755 MB/s
Running unrolled	...	204846 μ s		24.409 MB/s
Running reduced_dep	...	229905 μ s		21.748 MB/s
Running xbox	...	180256 μ s		27.738 MB/s
Running xbox_merged	...	143154 μ s		34.927 MB/s
Running zbb_default	...	203986 μ s		24.511 MB/s
Running zbb_xbox	...	145103 μ s		34.458 MB/s
--- 4 MB (4 MB, x2 repeats) ---				
Running default_loop	...	371702 μ s		21.523 MB/s
Running unrolled	...	332203 μ s		24.082 MB/s
Running reduced_dep	...	371888 μ s		21.512 MB/s
Running xbox	...	294034 μ s		27.208 MB/s
Running xbox_merged	...	235837 μ s		33.922 MB/s
Running zbb_default	...	331001 μ s		24.169 MB/s
Running zbb_xbox	...	240526 μ s		33.260 MB/s
--- 16 MB (16 MB, x1 repeats) ---				
Running default_loop	...	755323 μ s		21.183 MB/s
Running unrolled	...	679364 μ s		23.551 MB/s
Running reduced_dep	...	755850 μ s		21.168 MB/s
Running xbox	...	604573 μ s		26.465 MB/s
Running xbox_merged	...	496229 μ s		32.243 MB/s
Running zbb_default	...	674897 μ s		23.707 MB/s
Running zbb_xbox	...	503239 μ s		31.794 MB/s

Figure 13: 对比测试各种优化方案效率 (和上图同属一次测试, 1 MB 至 16 MB)

以默认循环实现 (default_loop) 为基准, 所有方案在每个数据规模下的运行时间 (elapsed_us)、吞吐率 (throughput_mbps) 和加速比 (speedup) 及其均值数据汇总如下。

Table 1: 各方案在不同数据规模下的运行时间（单位： μs ）

数据规模	运行时间（ elapsed_us ）						
	default_loop	unrolled	reduced_dep	xbox	xbox_merged	zbb_default	zbb_xbox
1 KB	88,985	78,646	88,667	68,855	53,654	78,304	54,257
16 KB	142,619	126,819	142,404	111,273	86,488	126,302	87,457
64 KB	142,590	126,666	142,267	110,963	86,442	126,072	87,641
256 KB	228,371	203,515	228,545	178,726	140,590	202,422	141,695
1 MB	229,836	204,846	229,905	180,256	143,154	203,986	145,103
4 MB	371,702	332,203	371,888	294,034	235,837	331,001	240,526
16 MB	755,323	679,364	755,850	604,573	496,229	674,897	503,239

Table 2: 各方案在不同数据规模下的吞吐率（单位：MB/s）

数据规模	吞吐率（ throughput_mbps ）						
	default_loop	unrolled	reduced_dep	xbox	xbox_merged	zbb_default	zbb_xbox
1 KB	21.949	24.834	22.028	28.366	36.402	24.943	35.998
16 KB	21.912	24.641	21.945	28.084	36.132	24.742	35.732
64 KB	21.916	24.671	21.966	28.163	36.151	24.787	35.657
256 KB	21.894	24.568	21.878	27.976	35.564	24.701	35.287
1 MB	21.755	24.409	21.748	27.738	34.927	24.511	34.458
4 MB	21.523	24.082	21.512	27.208	33.922	24.169	33.260
16 MB	21.183	23.551	21.168	26.465	32.243	23.707	31.794

Table 3: 各方案在不同数据规模下的加速比（基准：default_loop）

数据规模	加速比（ speedup, $\times$ vs baseline ）					
	unrolled	reduced_dep	xbox	xbox_merged	zbb_default	zbb_xbox
1 KB	1.1315	1.0036	1.2924	1.6585	1.1364	1.6401
16 KB	1.1246	1.0015	1.2817	1.6490	1.1292	1.6307
64 KB	1.1257	1.0023	1.2850	1.6495	1.1310	1.6270
256 KB	1.1221	0.9992	1.2778	1.6244	1.1282	1.6117
1 MB	1.1220	0.9997	1.2751	1.6055	1.1267	1.5840
4 MB	1.1189	0.9995	1.2641	1.5761	1.1230	1.5454
16 MB	1.1118	0.9993	1.2493	1.5221	1.1192	1.5009

为综合评价各优化方案在不同输入数据规模下的整体性能，进一步对表 3 中各方案的加速比计算几何平均值，结果如表 4 所示。与算术平均值相比，几何平均值更适合汇总不同测试规模下的相对性能提升，可以避免个别数据规模对整体结论产生过大的影响。

Table 4: 各方案在全部数据规模下的几何平均加速比

方案	default_loop	unrolled	reduced_dep	xbox	xbox_merged	zbb_default	zbb_xbox
几何平均加速比	1.0000	1.1224	1.0007	1.2750	1.6115	1.1277	1.5907

6 实验结果分析

6.1 总体性能对比

根据表3中加速比计算各方案在所有数据规模下的几何平均加速比, 结果如下:

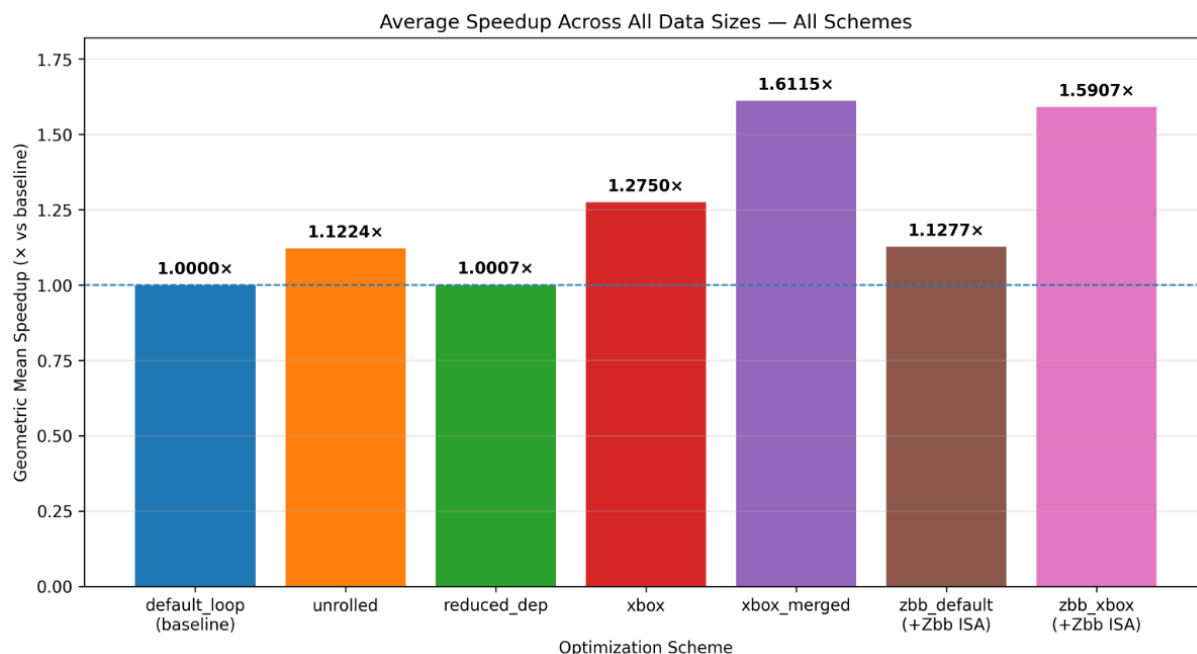


Figure 14: 各优化方案在所有数据规模下的几何平均加速比

从图14可以看出,6 种优化方案按照性能提升从低到高排序为:

`reduced_dep < unrolled < zbb_default < xbox < zbb_xbox < xbox_merged`

其中,xbox_merged的几何平均加速比为 1.6115x,是本实验中性能最优的实现;zbb_xbox的几何平均加速比为 1.5907x,略低于 xbox_merged; reduced_dep 的几何平均加速比仅为 1.0007x,与基线版本几乎相同。

实验结果表明,不同优化方案的效果存在明显差异。相比局部的指令级调整,将 SM4 轮函数中的部分计算转化为预计算查表操作能够获得更加显著的性能提升。

6.2 查表优化为何有效

查表优化包括 xbox 和 xbox_merged 两种方案。两种方案均利用了 SM4 线性变换的可加性,将部分运行时计算转化为预计算结果的组合。需要说明的是, xbox 并不是为完整的 32 bit 轮函数建立查找表。完整轮函数具有 2^{32} 种输入,不适合直接全部预计算。实际做法是:针对单个字节经过 S 盒替换和线性变换后的贡献,建立一张包含 256 个表项的查找表,再将 4 个字节对应的结果进行组合。以 1 MB 输入规模为例,三种方案的吞吐率如下:

Table 5: 1 MB 输入规模下查表优化方案的吞吐率对比

方案	吞吐率 (MB/s)	相对基线提升
default_loop	21.755	—
xbox	27.738	27.50%
xbox_merged	34.927	60.55%

xbox 方案通过查表减少了运行时的 S 盒替换、字节组合和循环移位相关操作，使吞吐率由 21.755 MB/s 提升至 27.738 MB/s。在此基础上，xbox_merged 进一步建立 4 张预旋转查找表，旋转偏移分别为 0 bit、8 bit、16 bit 和 24 bit。每张表包含 256 个 32 bit 表项，占用 1 KB 空间，总空间开销为 4 KB。程序在初始化阶段完成表项计算，进入加密过程后只需查表并完成异或组合，不再执行额外的运行时循环移位。从几何平均加速比看，xbox_merged 相较 xbox 又提升了约 26.39%。这说明在当前 RISC-V 开发板上，以少量额外存储空间换取消除运行时旋转操作具有较高收益。

6.3 微架构优化的有限效果

6.3.1 循环展开 (unrolled)

unrolled 方案通过将 SM4 的 32 轮迭代在编译阶段展开为 32 段顺序代码，减少了循环计数器自增、条件判断和分支跳转带来的额外开销。实验结果显示，unrolled 方案在全部数据规模下的几何平均加速比为 1.1224 $\times$ ，即相较默认循环版本提升约 12.24%。

该方案能够带来稳定收益，但提升幅度仍然有限。原因在于：

- SM4 的主要计算开销仍然集中在每轮迭代中的 S 盒替换、线性变换和状态更新。循环展开只能减少循环控制开销，无法消除轮函数本身的核心计算；
- 循环展开会增大代码体积，但开发板每核心具有 32 KiB 的 L1 指令缓存。结合实验结果来看，代码展开后并未出现明显的性能下降，说明增加的代码规模尚未对整体性能产生显著负面影响；
- 因此，循环展开适合作为一种较为稳定的基础优化方式，但难以单独突破 SM4 轮函数自身的计算瓶颈。

6.3.2 降低指令依赖 (reduced_dep)

reduced_dep 方案尝试通过重排异或操作来缩短原始表达式中的依赖链。默认实现中的连续异或表达式为：

$$X_1 \oplus X_2 \oplus X_3 \oplus rk[i]$$

优化后将其拆分为两个可以独立计算的中间结果：

$$tmp_1 = X_1 \oplus X_2$$

$$tmp_2 = X_3 \oplus rk[i]$$

$$tmp = tmp_1 \oplus tmp_2$$

理论上, 这种改写可以增加指令级并行机会, 减少连续异或之间的串行依赖。然而实验结果显示, `reduced_dep` 方案的几何平均加速比仅为 $1.0007\times$, 与默认循环版本基本相同。在 256 KB 及以上输入规模下, 其加速比甚至略低于 $1.00\times$ 。

这一现象可能由以下原因共同导致:

- 异或操作本身开销较低, 并不是 SM4 轮函数中的主要性能瓶颈。相比之下, S 盒访问和线性变换对整体运行时间的影响更加显著;
- 在 -O2 优化级别下, 编译器已经能够对简单异或表达式进行一定程度的重排和寄存器分配优化, 因此手工拆分表达式能够提供的额外收益较小;
- 新增的中间变量可能改变编译器的寄存器分配和指令调度结果, 从而抵消理论上的依赖链缩短收益;
- 当输入规模增大时, 流式数据访问和系统运行状态对总时间的影响更加明显, 微小的局部优化容易被这些额外开销稀释。

因此, `reduced_dep` 方案说明: 理论上可以增加并行机会的局部代码重排, 并不一定能够转化为可观测的整体性能提升。优化效果仍然取决于编译器行为、关键路径位置和具体硬件执行特征。

6.4 ISA 扩展的独立与叠加效应

RISC-V Zbb 位操作扩展提供了多种适合密码算法实现的位操作指令。其中, 与本实验中 SM4 实现关系较为密切的指令包括:

- `rev8`: 用于简化字节序反转操作。SM4 按照大端序读取和输出 32 位字, 而 RISC-V 开发板采用小端序存储。未启用 Zbb 扩展时, 字节序转换通常需要多次加载、移位和或运算; 启用 Zbb 扩展后, 编译器可以使用更紧凑的字节反转指令序列;
- `rori`: 用于简化循环移位操作。默认实现中的线性变换需要多次循环左移, 通常可拆解为移位与或运算的组合; 启用 Zbb 扩展后, 编译器可以使用旋转类指令减少相关指令开销。

需要说明的是, Zbb 扩展带来的实际收益不仅取决于指令集本身, 还取决于编译器是否能够识别相应代码模式并生成合适的目标指令。因此, 本实验通过对比不同编译配置下的实测结果评价其整体效果。

6.4.1 独立应用 Zbb 扩展

`zbb_default` 方案与 `default_loop` 方案采用相同的 32 轮循环结构, 区别在于编译时启用了 Zbb 相关扩展。两种方案的对比如下:

Table 6: 独立应用 Zbb 扩展前后的性能对比

方案	几何平均加速比
<code>default_loop</code>	$1.0000\times$
<code>zbb_default</code>	$1.1277\times$

实验结果显示,zbb_default 相对于基线版本取得了 $1.1277\times$ 的几何平均加速比,即整体性能提升约 12.77%。

这一结果说明,Zbb 扩展能够通过优化字节序转换和循环移位等局部操作带来稳定收益。但其提升幅度仍然明显低于 xbox 和 xbox_merged 等查表方案。这表明局部指令替换可以减少部分执行开销,但无法从根本上改变 SM4 轮函数的计算结构。

值得注意的是,zbb_default 的性能与循环展开方案 unrolled 较为接近。两者的几何平均加速比分别为 $1.1277\times$ 和 $1.1224\times$ 。这说明在当前开发板上,减少局部位操作开销与减少循环控制开销能够获得相近幅度的性能收益。

6.4.2 叠加应用 Zbb 扩展

zbb_xbox 方案在四张合并查找表的基础上进一步启用了 Zbb 扩展。为了分析 Zbb 扩展在查表优化完成后是否仍然具有明显的边际收益,应当将其与 xbox_merged 进行比较:

Table 7: 合并查表方案启用 Zbb 扩展前后的性能对比

方案	几何平均加速比
xbox_merged	$1.6115\times$
zbb_xbox	$1.5907\times$

实验结果显示,zbb_xbox 并未超过 xbox_merged,其几何平均加速比反而低约 1.29%。这说明在合并查表优化完成后,Zbb 扩展没有表现出可观测的额外正向收益。

其原因在于,xbox_merged 已经将四个字节位置对应的旋转结果预先存入四张查找表。预旋转操作在首次调用时的初始化阶段完成,加密主循环中只需执行查表和异或组合。因此,rori 等旋转类指令在主循环中已经缺少进一步发挥作用的空间。

在该方案中,Zbb 扩展仍可能优化分组加载和输出过程中的字节序转换,但这部分开销相对于 32 轮查表计算较小。同时,最终性能还会受到编译器指令选择、寄存器分配、代码布局和缓存访问行为等因素影响。因此,zbb_xbox 略低于 xbox_merged 并不意味着 Zbb 扩展本身必然产生负优化,而是说明其边际收益已经非常有限,并可能被其他微小波动抵消。

综上,Zbb 扩展适合用于优化默认循环实现,能够带来约 12.77% 的稳定提升;但在合并查表方案已经消除大部分运行时旋转操作后,Zbb 扩展与查表优化之间存在功能重叠,继续叠加的收益并不明显。由此可见,多种优化策略并不一定能够形成线性叠加效果。

6.5 数据规模、缓存与局部性

图 15展示了各优化方案的吞吐率随输入数据规模变化的趋势。

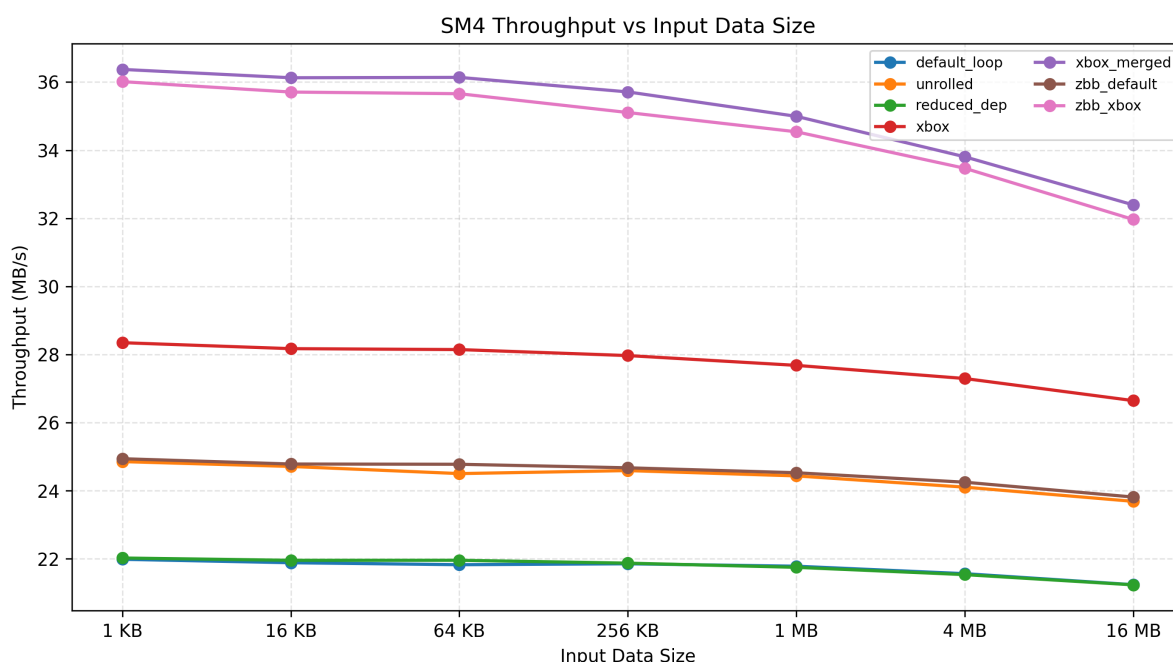


Figure 15: 各方案吞吐率随输入数据规模变化的趋势

结合图 15、表 2 和表 3, 可以观察到以下两个趋势。

1. 各方案的吞吐率均随输入规模增大而有所下降。

以默认循环方案 `default_loop` 为例, 吞吐率从 1 KB 输入规模下的 21.949 MB/s 下降至 16 MB 输入规模下的 21.183 MB/s, 下降约 3.49%。

查表方案的下降幅度更加明显。例如,

- `xbox_merged` 的吞吐率从 36.402 MB/s 下降至 32.243 MB/s, 下降约 11.43%;
- `zbb_xbox` 的吞吐率从 35.998 MB/s 下降至 31.794 MB/s, 下降约 11.68%。

2. 多数优化方案的加速比随输入规模增大而逐渐减小。

其中,

- `xbox_merged` 的加速比从 1 KB 输入规模下的 $1.6585\times$ 下降至 16 MB 输入规模下的 $1.5221\times$;
- `zbb_xbox` 则从 $1.6401\times$ 下降至 $1.5009\times$ 。

相比之下, `unrolled`、`zbb_default` 和 `reduced_dep` 等局部指令级优化方案的变化幅度较小。这说明查表方案虽然能够显著减少计算开销, 但其相对优势会随着输入规模增大而有所减弱。

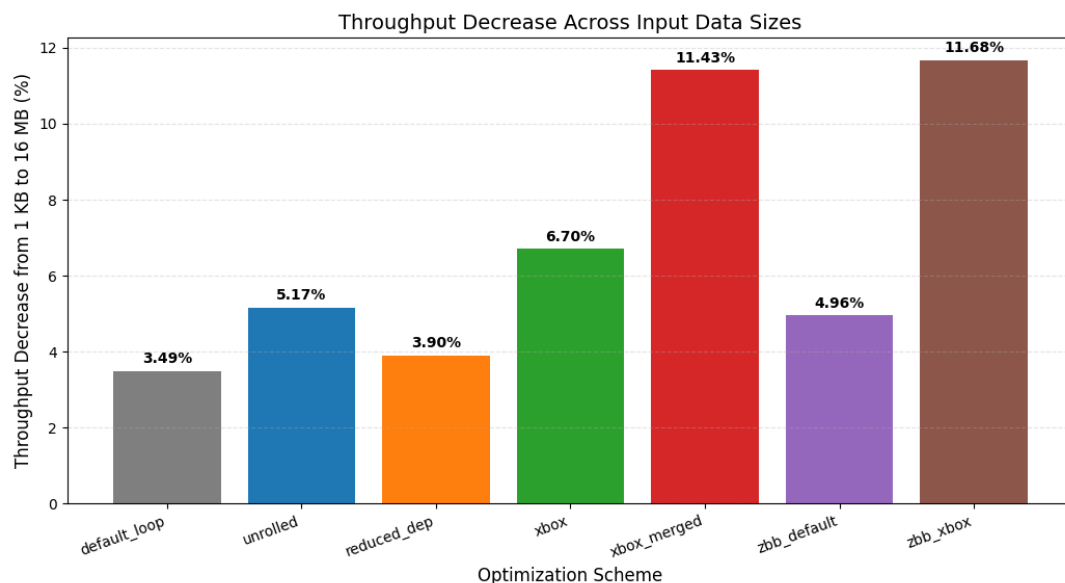


Figure 16: 各方案从 1 KB 到 16 MB 的吞吐率下降比例

上述现象可以结合局部性原理进行解释。

对于 `xbox_merged` 方案, 四张预计算查找表共占用 4 KB 空间。查找表会在每个明文分组和每轮加密中被反复访问, 因此具有较强的时间局部性。开发板每核心具有 32 KiB 的 L1 数据缓存, 从容量上看, 4 KB 查找表具备驻留高速缓存的条件。

另一方面, 输入数据按照分组顺序读取, 具有较好的空间局部性, 但单个明文分组处理完成后通常不会立即再次访问, 因此时间局部性较弱。随着输入规模从 1 KB 增长至 16 MB, 流式数据加载与写回开销在总运行时间中的占比逐渐增加, 从而稀释了计算优化带来的相对收益。

需要指出的是, 仅凭吞吐率和加速比曲线无法直接证明 Cache 缺失次数显著增加。更加严谨的结论是: 大规模输入下的性能衰减可能与缓存层次结构、流式数据访问开销和系统运行状态共同相关。若进一步使用硬件性能计数器统计 L1 Cache miss、L2 Cache miss 和内存访问次数, 可以更加准确地区分不同因素的影响。

7 实验总结

综合以上实验结果与分析, 可以得到以下结论:

1. 预计算查表是本实验中效果最显著的软件优化方式。

`xbox` 方案通过预计算单字节对轮函数的贡献, 取得了 $1.2750\times$ 的几何平均加速比; `xbox_merged` 进一步使用 4 张预旋转查找表消除运行时循环移位操作, 几何平均加速比达到 $1.6115\times$, 为本实验中性能最优的方案。这说明对于 SM4 轮函数, 采用空间换时间的方式减少运行时计算具有较高收益。

2. 局部指令级优化能够带来稳定但有限的性能提升。

循环展开方案 `unrolled` 和启用 Zbb 扩展的 `zbb_default` 分别取得了 $1.1224\times$ 和 $1.1277\times$ 的几何平均加速比。相比之下, `reduced_dep` 的几何平均加速比仅为 $1.0007\times$, 几乎没有可观收益。这说明局部代码重排或指令替换是否有效, 取决于其是否真正作用于程序的关键路径。

3. 不同优化策略叠加后并不一定产生线性增益。

`zbb_xbox` 在合并查表方案的基础上进一步启用了 `Zbb` 扩展, 但其几何平均加速比为 $1.5907\times$, 略低于 `xbox_merged` 的 $1.6115\times$ 。原因在于合并查表已经消除了大部分运行时循环移位操作, 使 `rori` 等位操作指令能够继续发挥作用的空间明显缩小。因此, 优化策略之间可能存在功能重叠, 实际效果还会受到编译器指令选择、寄存器分配和缓存访问行为等因素影响。

4. 输入规模增大后, 各方案吞吐率均有所下降。

其中, `default_loop` 从 1 KB 到 16 MB 的吞吐率下降约 3.49%, 而 `xbox_merged` 和 `zbb_xbox` 分别下降约 11.43% 和 11.68%。这说明查表方案虽然在全部测试规模下仍保持明显优势, 但对输入规模变化更加敏感。该现象可能与缓存层次结构、流式数据加载与写回开销以及系统运行状态共同相关。

5. 工程实现中应综合考虑性能收益与资源开销。

`xbox_merged` 需要额外存储 4 张查找表, 空间开销为 4 KB。对于本实验使用的 Orange Pi RV2 开发板, 每核心具有 32 KiB 的 L1 数据缓存, 因此该额外开销处于可接受范围。在类似资源条件下, 建议优先采用 `xbox_merged` 方案; 在缓存和存储空间更加受限的平台上, 则应根据实际资源约束在性能与空间之间进行权衡。